TRUSTEANDO

A Decentralised Verifiable Knowledge Graph Protocol

confidencenode.org/protocolos/trusteando
Autor: confidencenode.org/members/confidencenode0

18 de Marzo de 2026 — v0.1

Abstract

Trusteando is an open protocol for a decentralised, cryptographically verifiable Knowledge Graph. Entities — universities, companies, professional bodies, individuals — are nodes identified by the hash of their public URL. Relations between nodes are expressed as folder structures published by each entity in its own web space. The folder structure is the credential: there is no credential information outside it. Any entity with a web presence can act as an identity issuer for the entities it recognises, using a chain of shared keys established via Diffie-Hellman key exchange, with authorship guaranteed by ECDSA signatures. The graph is dynamic: temporal dimension is encoded in desde/hasta folder conventions, giving the graph full verifiable history. Identity is portable across migrations via an emergency key mechanism — the receiving entity publishes and signs the full chain of prior identities in its own old_identities table, outside the control of the migrating node. A root node bootstraps the system; over time, high-reputation nodes become independent trust anchors, making the system polycentric by design. When the root is unavailable, any sufficiently reputable node can perform its verification functions, since the root holds no privileged state — it only executes a public algorithm. Membership precedes recognition. Contextual trust suffices for most verification. Cryptographic verification handles the rest.

1. Introducción

Los sistemas de identidad digital existentes comparten un defecto estructural: requieren que la identidad sea concedida por una autoridad central antes de poder existir. Los certificados X.509 dependen de Autoridades de Certificación comerciales. Los sistemas de identidad federada como OAuth delegan la identidad a proveedores privados — Google, Apple, Microsoft — que pueden revocarla unilateralmente. Los Knowledge Graphs existentes como Wikidata o Google Knowledge Graph son centralizados o requieren consenso de un editor central para escribir.

Trusteado parte de una observación diferente: la identidad de una entidad real no necesita ser concedida — ya existe. Una universidad existe. Tiene una URL. Tiene personal. Tiene autoridad real sobre ese personal. Las relaciones entre entidades — quién pertenece a qué, desde cuándo, con qué rol — también existen. Lo que falta no es un mecanismo para crear esa identidad sino un protocolo para expresarla como un grafo de conocimiento verificable, descentralizado, y con memoria temporal.

Trusteando es ese protocolo. Los nodos del grafo son entidades identificadas por el hash de su URL pública. Las aristas son relaciones expresadas como estructuras de carpetas publicadas por cada entidad en su propio espacio web. La firma criptográfica de cada entidad sobre su propio espacio garantiza la integridad. La cadena de confianza entre niveles permite la verificación sin depender de ningún servidor central en el uso cotidiano.

2. Principios de diseño

2.1 La URL como ancla de identidad

El identificador de cualquier entidad en el sistema es el hash criptográfico de su URL canónica:

```
entity_id = SHA-256(canonical_url)
```

La entidad no necesita registrarse en ningún sistema previo. Su URL ya existe y es pública. Trusteado la reconoce como ancla de identidad sin añadir fricción. Dos entidades distintas tienen distintas URLs y por tanto distintos identificadores — no hay colisión posible bajo SHA-256 en la práctica.

La URL canónica se define como la URL de la página principal de la entidad con esquema https, sin barra final, en minúsculas. Por ejemplo: https://universidad.es

2.2 La pertenencia precede al reconocimiento

Si una universidad publica en su propia web — por ejemplo en https://universidad.es/trusteado — el hash de un profesor junto con su clave pública, ese profesor ya pertenece al espacio de identidad de esa universidad. La cadena de confianza existe antes de que el nodo raíz la valide formalmente.

El nodo raíz no crea la relación entre la universidad y su profesor — la eleva a confianza criptográficamente verificable por terceros que no conocen a ninguna de las partes. Para el verificador que ya conoce y confía en la universidad, la publicación en su URL es suficiente.

2.3 La confianza contextual es suficiente para el uso cotidiano

Ver el hash de Juan Escobar publicado en universidad.es/profesores ya transmite confianza a cualquier verificador que conozca esa universidad. No hace falta que el nodo raíz lo certifique. La universidad tiene autoridad real — su URL es su identidad, su reputación institucional es su aval.

La verificación criptográfica completa resuelve los casos donde la confianza contextual no alcanza: cuando el verificador no conoce la entidad, cuando la verificación debe ser automática, o cuando hay disputa sobre la autenticidad de una credencial.

2.5 La autoridad se demuestra controlando la URL

La autoridad de una entidad para emitir credenciales no se declara — se demuestra controlando la URL bajo la que publica. Nadie puede publicar bajo universidad.es/profesores/ sin controlar universidad.es. El DNS y el servidor web son la prueba de control, exactamente igual que en HTTPS.

Esto resuelve la pregunta de cómo sabe un verificador que una entidad tiene autoridad para emitir una credencial: porque la credencial está publicada bajo una URL que esa entidad controla. No hace falta ninguna declaración adicional. El control es la autoridad.

```
confidencenode.org                                  ← ConfidenceNode
controla esto
  /protocolos/trusteado                            ← por tanto controla
esto
  /members/confidencenode0                         ← y esto
  /members/confidencenode0/externals/github
    → github.com/ConfidenceNode0                   ← presencia externa
vinculada

universidad.es                                      ← la universidad
controla esto
  /profesores/juan-ruiz                            ← por tanto controla
esto
  /old_identities/                                 ← y esto
```

La carpeta externals permite a una entidad vincular desde su espacio propio cualquier presencia en sistemas externos que no controla — GitHub, ORCID, LinkedIn, rankings, registros oficiales. El vínculo lo firma la propia entidad, así que la autoridad sobre el vínculo es suya. Pero externals es semántica del protocolo, no una convención de nomenclatura: declara explícitamente que la entidad no tiene autoridad sobre lo que hay en el destino. Un verificador que lee externals/github sabe que la entidad vincula algo que no controla y que por tanto no puede garantizar su contenido — solo garantiza que el vínculo es auténtico.

Esta distinción contrasta directamente con el resto de carpetas, donde la ausencia de externals implica control:

```
/members/confidencenode0/externals/github
  → la entidad declara que NO controla el destino

/protocolos/trusteado
  → la entidad declara implícitamente que SÍ controla esto

Otros ejemplos válidos:
  /members/confidencenode0/externals/orcid
  /members/confidencenode0/externals/linkedin
  universidad.es/externals/ministerio-educacion
  universidad.es/externals/rankings/qs-world
```

En Trusteado la estructura de la URL declara la autoridad. La carpeta externals es la única excepción explícita — y su existencia hace que la regla general sea aún más clara.

Este mecanismo ilustra un caso real: confidencenode.org/members/confidencenode0/externals/github apunta a github.com/ConfidenceNode0, donde vive el código del protocolo. El dominio propio es la identidad canónica; GitHub es el hosting actual del código. Cuando el hosting propio esté levantado, el código migrará — pero la identidad canónica no cambia.

2.6 La regla de oro — la estructura de carpetas es la credencial

Toda la información que una entidad está obligada a ofrecer sobre credenciales está almacenada en su estructura de carpetas públicas. No hay base de datos oculta, no hay campos en documentos separados, no hay APIs con estado interno que un verificador necesite consultar.

Un verificador puede comprobar cualquier credencial siguiendo únicamente URLs públicas. Si la información no está en la estructura de carpetas, no existe como credencial del protocolo. Esta regla no es una convención — es una condición de validez.

Las consecuencias prácticas son profundas: verificabilidad total sin cooperación de la entidad emisora, auditabilidad completa del espacio de identidad de cualquier entidad, y resiliencia ante caídas — una copia estática de las carpetas es suficiente para reconstruir toda la información.

2.7 Trusteando como Knowledge Graph descentralizado

El conjunto de entidades y relaciones que el protocolo expresa forma un Knowledge Graph con tres propiedades que los sistemas existentes no combinan:

- Descentralizado — cada nodo escribe en su propio espacio. Nadie necesita permiso de un editor central para publicar relaciones.
- Verificable criptográficamente — cada arista está firmada por quien la emite. No hay que confiar en ningún editor central — la firma lo prueba.
- Dinámico con historial completo — las carpetas desde/hasta dan dimensión temporal al grafo. No es una foto estática sino un grafo que evoluciona con historia íntegra y verificable.

Las credenciales son aristas del grafo. Las entidades son nodos. La cadena de confianza es la topología. El protocolo de identidad es un caso particular de este grafo más general.

2.8 El protocolo es abierto e irrevocablemente público

El formato del protocolo, la capacidad de montar nodos propios, y la verificación manual consultando URLs directamente son libres y permanecen así. Ninguna entidad — incluido el nodo raíz de Trusteado — puede impedir a nadie implementar el protocolo, verificar credenciales, o publicar su propio espacio de identidad.

Esta apertura no es una concesión — es lo que da valor al sistema. Un protocolo que puede ser revocado por su creador no es infraestructura: es un servicio con condiciones de uso.

3. Estructura de una credencial

Una credencial Trusteado tiene tres campos:

Campo	Descripción
public_key	Clave pública del firmante. Identifica al emisor de la credencial de forma verificable.
hash(mensaje)	Hash SHA-256 del contenido que se afirma. El contenido puede mantenerse privado — solo el hash es necesario para la verificación.
hash_verificacion	HMAC-SHA-256 del hash(mensaje) usando la clave compartida

	entre el firmante y su nivel superior. Solo el nivel superior puede verificarlo — confirma que la credencial pertenece a alguien reconocido por ese nivel.
auth_signature	Firma ECDSA del msg_hash con la clave privada del firmante. Solo el poseedor de esa clave privada puede generarla. Cualquier tercero puede verificarla con la clave pública del firmante. El root guarda un compromiso firmado en el alta que vincula esta clave con la identidad real del firmante, permitiendo resolución de disputas con respaldo legal.

La credencial no es un documento — es la presencia de un nodo en una carpeta del espacio de identidad de su superior. La estructura de carpetas es la credencial. Por ejemplo:

```
universidad.es/profesores/juan-escobar          ← Juan es Profesor
universidad.es/doctores/juan-escobar            ← Juan es Doctor
universidad.es/profesores/desde/2021-09-01/juan-escobar   ←
incorporación
universidad.es/profesores/hasta/2024-06-30/juan-escobar   ← baja
```

La dimensión temporal está codificada en la propia estructura de carpetas — bajo control de la universidad, no de Juan. Juan no puede alterar las fechas de incorporación ni de baja. La revocación de una credencial específica es simplemente eliminar al nodo de la carpeta correspondiente, sin afectar a las demás.

Adicionalmente, cada credencial puede incluir un bloque de metadatos firmados:

```
{
  "issuer":         "SHA-256(url_del_emisor)",
  "subject":        "SHA-256(url_o_identificador_del_sujeto)",
  "public_key":     "base64(clave_publica_del_emisor)",
  "msg_hash":       "SHA-256(contenido_de_la_credencial)",
  "hmac":           "HMAC-SHA-256(msg_hash, clave_compartida)",
  "auth_signature": "ECDSA.sign(msg_hash, clave_privada_firmante)",
  "timestamp":      1710000000
}
```

El mensaje cuyo hash se incluye puede tener cualquier estructura. El protocolo no impone un esquema. Ejemplos de mensajes válidos:

```
{ "role": "Profesor Asociado", "department": "Informatica" }
{ "fact": "email_sent", "from": "a@uni.es", "to": "b@uni.es", "date":
"2026-03-17" }
{ "claim": "age_over_18", "verified_by": "SHA-256(universidad.es)" }
{ "presence": "SHA-256(https://venue.es)", "timestamp": 1710000000 }
```

4. La cadena de confianza

4.1 Estructura general

El sistema organiza la confianza en niveles. Cada nivel puede emitir credenciales sobre los niveles que reconoce. La cadena tiene la siguiente forma general:

```
Trusteado root
  └─ reconoce a universidad.es
       └─ reconoce a juan@universidad.es (Profesor Asociado)
            └─ juan firma credenciales verificables

Trusteado root
  └─ reconoce a empresa.com
       └─ reconoce a correo.empresa.com (servidor de correo)
            └─ correo certifica comunicaciones entre empleados
```

El número de niveles no está limitado por el protocolo. Cada entidad decide con qué profundidad organiza su espacio de identidad interno.

4.2 Establecimiento de la clave compartida

Cada par de niveles adyacentes comparte una clave simétrica establecida mediante intercambio Diffie-Hellman sobre curva elíptica (ECDH). Este es el mismo mecanismo que usa TLS en cada conexión HTTPS — probado, estandarizado, y bien comprendido.

El proceso de establecimiento ocurre una sola vez por par:

```
# La entidad inferior genera su par de claves efímero
privada_inferior, publica_inferior = ECDH.generate_keypair(curve=P-
256)

# El nivel superior genera su par efímero para este intercambio
privada_superior, publica_superior = ECDH.generate_keypair(curve=P-
256)

# Intercambian claves públicas (canal autenticado)

# Cada parte deriva la misma clave compartida independientemente
clave_compartida = ECDH.derive(privada_inferior, publica_superior)
clave_compartida = ECDH.derive(privada_superior, publica_inferior)
# → mismo resultado en ambos lados

# La clave compartida se usa para generar y verificar HMAC
hash_verificacion = HMAC-SHA-256(msg_hash, clave_compartida)
```

La propiedad fundamental del intercambio Diffie-Hellman es que ninguna de las dos partes conoce la clave privada de la otra, y nadie que observe el intercambio de claves públicas puede derivar la clave compartida. El nivel superior no puede impersonar al inferior porque no tiene su clave privada.

El root no conoce la clave compartida entre la universidad y Juan. La universidad no conoce la clave compartida entre el root y otras entidades. Cada par tiene una clave que es exclusivamente suya.

4.3 Verificación en cadena

Para verificar que una credencial firmada por Juan como Profesor Asociado es válida, el verificador sigue estos pasos:

```
1. Obtener la credencial de Juan:
   (public_key_juan, msg_hash, hmac, auth_signature)
```

```
2. Consultar universidad.es/trusteado — ¿está public_key_juan
registrada?
   → sí: Juan pertenece al espacio de identidad de la universidad

3. Verificar AUTENTICIDAD — pedir a la universidad que verifique hmac:
   universidad comprueba: HMAC(msg_hash, clave_compartida_juan_uni) ==
hmac
   → la credencial viene de alguien reconocido por la universidad

4. Verificar AUTORÍA — con la clave pública de Juan:
   ECDSA.verify(msg_hash, auth_signature, public_key_juan) == válido
   → solo Juan pudo generar esta firma

5. ¿Es universidad.es un agente reconocido?
   → consultar registro público del root
   → root firmó: SHA-256(universidad.es) → public_key_universidad
   → válido

6. Credencial verificada — autenticidad y autoría confirmadas.
```

El verificador no necesita conocer la clave compartida entre Juan y la universidad — solo necesita que la universidad confirme la validez del HMAC. La universidad no revela la clave compartida: solo responde sí o no a la consulta de verificación.

4.4 Separación entre autenticidad y autoría

La introducción de auth_signature resuelve un problema estructural importante: sin ella, el nivel superior podría suplantar al inferior. Conoce la clave compartida, puede generar un HMAC válido, y podría construir una credencial aparentemente legítima atribuida a Juan. Con auth_signature eso es imposible — la clave privada de Juan nunca sale de su dispositivo.

Las dos propiedades son verificables independientemente:

- Autenticidad — verificada por el nivel superior mediante el HMAC. Responde a: ¿viene esta credencial de alguien que yo reconozco?
- Autoría — verificada por cualquier tercero con la clave pública del firmante mediante ECDSA. Responde a: ¿fue específicamente esta persona y no otra?

Esta separación hace la cadena de confianza robusta frente a compromisos internos: incluso si un nivel superior es malicioso o está comprometido, no puede fabricar credenciales atribuibles a sus agentes inferiores.

4.5 Non-repudiation y resolución de disputas

La firma ECDSA establece la evidencia técnica de autoría. Sin embargo el non-repudiation completo — la imposibilidad de negar la autoría incluso cuando ninguna de las partes quiere asumir responsabilidad — requiere algo más que criptografía.

En el momento del alta, el firmante suscribe ante el root un compromiso de responsabilidad: acepta que la clave privada registrada es suya y que asume responsabilidad por las credenciales firmadas con ella. El root custodia ese compromiso.

En caso de disputa en que ninguna de las partes quiere asumir la autoría:

```
1. El root presenta el compromiso firmado por Juan en el alta
2. La evidencia criptográfica vincula ese compromiso con la credencial
disputada
   mediante auth_signature verificable con la clave pública registrada
```

```
3. La responsabilidad queda establecida aunque Juan no quiera
reconocerla
```

Este mecanismo reconoce explícitamente que el non-repudiation no es solo un problema criptográfico. La criptografía establece la evidencia — el marco legal le da fuerza vinculante. Es el mismo modelo que usan los sistemas de firma electrónica cualificada bajo el reglamento eIDAS en la Unión Europea. El protocolo establece la evidencia; el marco institucional le da fuerza. Ambas capas son necesarias y ninguna sustituye a la otra.

4.6 El rol del nodo raíz

El root tiene tres responsabilidades:

- Una vez por agente: establecer la clave compartida con la entidad mediante ECDH, y publicar en su registro la asociación entre el entity_id de la entidad y su clave pública.
- Mantener un registro público firmado de agentes reconocidos, actualizado periódicamente con un TTL que permite a los verificadores cachearlo localmente.
- Custodiar las claves de emergencia registradas y aprobar migraciones de identidad cuando se le presenta evidencia válida.

El registro de claves de emergencia con el root es asíncrono — un nodo puede publicar su hash_publico en su propia web y operar con normalidad antes de que el root lo registre. El root procesa los registros según su capacidad. El sistema no se bloquea mientras tanto.

4.7 Resiliencia de identidad — la clave de emergencia

Cada nodo genera en el momento de su creación una clave de emergencia que nunca comparte con su superior ni con ningún otro nodo. Con ella calcula su hash_publico:

```
clave_emergencia  ← generada localmente, nunca compartida con el
superior
hash_publico = hash(entity_id, clave_emergencia)
```

El hash_publico se publica en la propia web del nodo como parte de su identidad pública, en una ruta estándar que cualquier implementación del protocolo sabe dónde buscar:

```
universidad.es/profesores/juan-escobar/hash_publico/a3f9e2b1c4d7...
```

Cualquier tercero puede ver este valor. Solo Juan puede demostrar que lo generó, porque solo él conoce la clave_emergencia que lo produce. El hash_publico está anclado en dos lugares independientes: la propia web del nodo y, cuando el root lo registra, el registro central. Si cualquiera de los dos falla, el otro sostiene la cadena.

Migración voluntaria a un nuevo superior

Cuando Juan quiere migrar a una nueva universidad, el proceso es:

```
1. Juan revela su clave_emergencia a universidad-b.es

2. Universidad-b verifica contra el hash_publico publicado por Juan:
   hash(entity_id_juan, clave_emergencia) == hash_publico
   → la adopción es legítima y voluntaria — Juan entregó la clave
```

```
3. Universidad-b comunica al root que conoce la clave_emergencia de
Juan.
   Root aprueba la migración.

4. Universidad-b crea la entrada de Juan en su espacio:
   universidad-b.es/profesores/juan-escobar/

5. Universidad-b registra el vínculo en su propia tabla
old_identities,
   bajo su control — no bajo el control de Juan:
   universidad-b.es/old_identities/
     hash_nuevo → [ hash_nuevo, hash_antiguo_1, hash_antiguo_2, ... ]

   La cadena completa de identidades anteriores se publica entera.
   Universidad-b la recibió de universidad-a en el proceso de
adopción,
   añade la nueva entrada, y la firma con su propia clave.

6. Juan genera una nueva clave_emergencia y publica nuevo
hash_publico.
   La clave anterior queda invalidada.
```

La tabla old_identities es un registro institucional de la universidad, no de Juan. Juan no puede modificarla ni omitir entradas. Está firmada por la universidad con la misma autoridad que cualquier otra credencial que emite.

Publicar la cadena completa — no solo el vínculo inmediato anterior — permite a cualquier agente del sistema verificar el historial completo de Juan con una sola consulta, sin tener que saltar de servidor en servidor siguiendo eslabones uno a uno. Cada migración extiende la cadena por un elemento; la universidad receptora la hereda, la extiende, y la publica entera firmada.

Adopción temporal por el root cuando el superior desaparece

Si el superior de Juan desaparece — la universidad cierra, pierde el dominio, o es revocada — Juan puede identificarse ante el root con su clave_emergencia:

```
1. Juan presenta al root su clave_emergencia
2. Root verifica: hash(entity_id_juan, clave_emergencia) ==
hash_publico registrado
3. Root acoge a Juan como nodo huérfano bajo su tutela directa
4. Juan opera con normalidad mientras encuentra un nuevo superior
5. Cuando un nuevo superior lo adopta, sigue el proceso de migración
estándar
```

Esta propiedad garantiza que la identidad de un nodo sobrevive a la desaparición de su superior. No empieza de cero — continúa con su historial, sus credenciales anteriores verificables, y su reputación acumulada. Es análogo a la portabilidad del número de teléfono, pero para identidad criptográfica: el titular controla la portabilidad, no el operador.

Degradación con gracia

Si el root está saturado o no disponible, los nodos existentes siguen operando con normalidad. Las verificaciones no requieren al root — consultan directamente las webs de las entidades. Solo se bloquean los registros nuevos y las migraciones, no el uso cotidiano del sistema.

4.9 Raíces de confianza distribuidas

Con el tiempo, las entidades con reconocimiento público suficiente se convierten en raíces de confianza naturales. Un verificador que conoce y confía en una universidad puede verificar credenciales de sus profesores sin consultar al root de Trusteado.

El sistema evoluciona de jerárquico a policéntrico de forma orgánica. El root de Trusteado hace bien su trabajo cuando deja de ser necesario para el uso cotidiano — cuando la red de confianza tiene suficientes nodos con reputación propia para sostenerse sin una autoridad central única.

5. Dos niveles de existencia

Una entidad puede operar en dos niveles distintos en el sistema:

Nivel	Descripción
Identidad declarada	La entidad publica sus hashes y claves en su propia URL. La cadena es coherente y verificable por cualquiera que conozca la entidad. No hay garantía externa para verificadores que no la conocen. Útil para sistemas internos, comunidades cerradas, o para construir la cadena antes de solicitar reconocimiento formal.
Identidad reconocida	El root ha establecido una clave compartida con la entidad y publicado su certificado de agente. La cadena es verificable por cualquier tercero sin conocimiento previo de la entidad. La confianza es transitiva hacia los niveles inferiores.

Una entidad puede operar en nivel 1 indefinidamente. El nivel 2 añade verificabilidad para terceros desconocidos — no es un requisito para que el sistema funcione.

6. Privacidad y divulgación selectiva

6.1 Privacidad por defecto

El verificador ve solo el resultado de la verificación: la credencial es válida o no lo es. La cadena completa de confianza — qué entidades están implicadas, qué dice el mensaje original — queda oculta salvo que el titular de la credencial decida revelarla explícitamente.

El hash del mensaje permite verificar la integridad sin revelar el contenido. El emisor puede demostrar que el mensaje no ha sido alterado sin mostrar el mensaje mismo.

6.2 Divulgación selectiva

El titular de una credencial puede elegir qué revelar en cada contexto:

- Solo el resultado: la credencial es válida — sin revelar quién la emitió ni qué dice
- El emisor pero no el contenido: esta credencial viene de universidad.es — sin revelar el rol específico
- La cadena completa: universidad.es certifica que Juan Escobar es Profesor Asociado del Departamento de Informática

Esta propiedad permite casos de uso como verificación de mayoría de edad sin revelar la fecha de nacimiento, o verificación de pertenencia a una organización sin revelar el cargo.

6.3 Lo que el sistema no oculta

El hecho de que una entidad está registrada como agente en el root es público — forma parte del registro de agentes reconocidos. El entity_id (hash de la URL) de una entidad es derivable por cualquiera que conozca su URL. El protocolo no está diseñado para ocultar la existencia de las entidades participantes, solo para proteger el contenido de las credenciales que emiten.

7. Casos de uso

El protocolo es agnóstico al contenido de las credenciales. Los casos de uso siguientes ilustran la generalidad del mecanismo — no son los únicos posibles.

7.1 Credenciales académicas y profesionales

Una universidad registra a sus profesores en su espacio de identidad. Un colegio de médicos registra a sus colegiados. Un bar de abogados registra a sus miembros. Cualquier verificador puede comprobar la credencial sin llamar a ningún servidor central — consultando la URL de la entidad emisora y verificando la firma.

7.2 Reseñas verificadas por presencia

Un establecimiento registra tokens de presencia temporales — por ejemplo mediante un código QR rotativo disponible solo en el local. Un usuario que estuvo físicamente en el local puede obtener un token de presencia y usarlo para firmar una reseña. La reseña lleva una credencial de presencia que cualquiera puede verificar: esta persona estuvo en este lugar.

> *Este es el caso de uso que motivó el desarrollo inicial del protocolo. La verificación de presencia es un caso particular de credencial verificable — no un mecanismo distinto.*

7.3 Verificación de comunicaciones

El servidor de correo de una empresa, registrado como agente delegado por la empresa, puede emitir credenciales que certifiquen que un correo fue enviado entre dos partes en una fecha determinada. El contenido del correo puede permanecer privado — solo el hash del mensaje y la credencial de envío son necesarios para la verificación.

7.4 Verificación de condición sin revelar datos

Una entidad con autoridad sobre datos personales — un registro civil, una universidad, un sistema de salud — puede emitir credenciales de condición: mayor de edad, titulado universitario, profesional sanitario colegiado. El verificador recibe la confirmación de la condición sin acceder a los datos subyacentes.

7.5 Delegación de autoridad dentro de organizaciones

Una empresa puede registrar como agentes delegados sus divisiones, departamentos, o sistemas internos. Cada uno puede emitir credenciales en su ámbito. La cadena de delegación es verificable externamente sin exponer la estructura interna de la organización.

7.6 Autoría verificable de contenido

Un autor puede firmar un documento, un artículo, o cualquier contenido digital con su credencial Trusteado. Cualquier verificador puede comprobar que el contenido fue producido por esa persona y que no ha sido alterado desde entonces, sin necesidad de terceros de confianza adicionales.

8. Modelo de sostenibilidad

El protocolo es abierto. La implementación de referencia es software libre. Lo que puede monetizarse es la capa de servicio sobre el protocolo — no el protocolo mismo.

8.1 Reconocimiento formal como agente

El registro en el root de Trusteado tiene valor de señal, especialmente en las fases iniciales cuando el root es la principal raíz de confianza establecida. El reconocimiento formal tiene un coste razonable y criterio público y transparente.

8.2 Infraestructura gestionada

Implementar un nodo Trusteado completo requiere conocimientos técnicos que no todas las entidades tienen. Trusteado ofrece infraestructura gestionada — el nodo como servicio — para entidades que quieren participar sin gestionar la infraestructura. Es el modelo de WordPress: el protocolo es libre, el hosting gestionado no tiene por qué serlo.

8.3 API de verificación

Aplicaciones de terceros que quieran integrar verificación Trusteado sin implementar el protocolo completo pueden usar una API de verificación. El precio es por volumen de verificaciones. El protocolo permanece libre — la API es una conveniencia, no un requisito.

8.4 Lo que permanece libre

- El formato completo del protocolo
- La capacidad de montar nodos propios
- La verificación manual consultando URLs directamente
- La publicación de credenciales sin pasar por el root
- La implementación de verificadores propios

9. El problema del arranque

El valor del root depende de la reputación de quien lo opera. Los sistemas de confianza no nacen de la nada — necesitan un punto de partida con credibilidad suficiente para que las primeras entidades quieran registrarse.

Este es el mismo problema que resolvieron DNS, Let's Encrypt, y cualquier PKI: al principio la autoridad no viene del sistema — viene de las personas y organizaciones detrás de él. La solución no es técnica sino social: las primeras entidades reconocidas se consiguen con relaciones personales, casos de uso concretos que resuelven problemas reales, y credibilidad del equipo fundador en ese contexto.

La estrategia de arranque correcta es publicar el protocolo abiertamente y con prioridad temporal establecida, antes de que ningún competidor pueda reclamar la misma idea. La apertura no debilita la posición del root — la refuerza. Un root que opera un protocolo cerrado puede ser reemplazado por cualquiera que publique un protocolo mejor. Un root que opera el protocolo estándar de facto acumula legitimidad que no se puede copiar.

9.1 Resistencia a Embrace, Extend, Extinguish

La estrategia conocida como Embrace, Extend, Extinguish consiste en adoptar un estándar abierto, añadir extensiones propietarias que crean dependencia, y finalmente hacer irrelevante el estándar original. Es el patrón que ha eliminado o debilitado múltiples protocolos abiertos a lo largo de la historia de internet.

Trusteando incorpora resistencia a este patrón por diseño:

- Embrace — cualquiera puede adoptar el protocolo. Inevitable y deseable.
- Extend — cualquier extensión propietaria debe declararse bajo trusteado/extensions/ con justificación. Las extensiones no estándar no son interpretadas por otros parsers. La fragmentación es visible y auditable.
- Extinguish — los datos viven en la URL de cada entidad, no en ningún servicio central. No hay nada que apagar. GPL v3 impide distribuir versiones cerradas. El root no guarda estado — no se puede monopolizar.

La defensa más efectiva contra este patrón es cubrir el espacio funcional tan completamente que cualquier extensión propietaria sea marginal o redundante. El vocabulario extenso de carpetas estándar — features/, payments/, messaging/, hooks/, media/ — no es complejidad innecesaria: es territorio que no se puede usar como cuña.

9.2 Impacto en servicios intermediarios

Un efecto secundario del protocolo es que la información estructurada de cualquier negocio — horarios, ubicación, características, reputación verificada — pasa a ser pública y accesible directamente desde su URL. Esto permite construir servicios de búsqueda, comparación, y agregación sobre datos abiertos sin depender de intermediarios que actualmente centralizan y monetizan esa información.

El protocolo no elimina esos servicios — simplemente hace que su valor resida en la capa de presentación y experiencia de usuario, no en el acceso exclusivo a los datos. Los hooks y proxies del protocolo garantizan además que el negocio nunca esté atado a ningún proveedor de servicios por razones técnicas — cambiar de proveedor es siempre una decisión de negocio, nunca una migración técnica.

10. Dos capas — registro y trámite

El protocolo distingue explícitamente dos capas que en los sistemas actuales se confunden:

10.1 La capa de registro — el Knowledge Graph

La capa de registro es el grafo permanente de hechos verificados. Lo que es cada nodo y qué relaciones tiene. Cada entrada es firmada por la entidad que la publica, inmutable una vez establecida, y auditable por cualquier tercero sin cooperación de nadie.

```
universidad.es/profesores/juan-ruiz          ← hecho verificado,
permanente
```

```
estados/españa/ciudadanos/juan-ruiz          ← hecho verificado,
permanente
colegios/medicos/colegiados/juan-ruiz        ← hecho verificado,
permanente
```

La redundancia en esta capa es una propiedad positiva — que varios nodos independientes certifiquen lo mismo sobre Juan añade resiliencia. Si dos nodos de alto grado dicen cosas contradictorias, esa inconsistencia es una señal relevante por sí misma.

10.2 La capa de trámite — los flujos entre nodos

La capa de trámite son los intercambios de información entre nodos para llegar a establecer hechos en la capa de registro. Son efímeros, operacionales, y no forman parte del grafo permanente. Representan el proceso, no el resultado.

```
Universidad pregunta a estado: ¿juan-ruiz tiene DNI válido?   ← flujo,
efímero
Estado responde: sí, expedido 2015                            ← flujo,
efímero
Universidad añade juan-ruiz a /profesores/desde/2026-03-17/   ← entra
al grafo
```

El protocolo puede definir el formato estándar de estos flujos para facilitar la interoperabilidad entre implementaciones. Pero los flujos no son el grafo — son las aristas operacionales que permiten construirlo. Un verificador externo nunca necesita ver los flujos, solo el resultado en el grafo.

10.3 Identidad civil como condición mínima

Un nodo puede existir técnicamente sin identidad civil verificable. Pero para que un nodo represente a una persona física real, el protocolo requiere al menos una credencial emitida por una entidad con autoridad civil reconocida — un estado, un registro oficial — que vincule ese nodo con una identidad legal.

La estructura natural para personas físicas es:

```
estados/españa/ciudadanos/juan-ruiz     ← ancla de identidad civil
  → universidad certifica: juan-ruiz es Profesor
  → colegio certifica: juan-ruiz está colegiado
  → empresa certifica: juan-ruiz es empleado
```

El nodo de identidad civil es el ancla. Las credenciales son aristas del grafo que apuntan a ese nodo desde distintas entidades. Una persona puede tener múltiples credenciales de múltiples entidades — todas apuntan al mismo nodo de identidad civil.

11. Especificación declarativa e implementación funcional

El protocolo distingue dos niveles que son independientes y complementarios:

11.1 La especificación declarativa — las carpetas

Las carpetas definen qué existe y qué significa. Es el contrato público del protocolo — el QUÉ. Cualquiera puede leerlo, verificarlo, y auditarlo sin herramientas especiales ni conocimiento de la implementación interna. La estructura de carpetas es permanente, firmada, y auditable.

11.2 La implementación funcional — la base de datos

La base de datos, la API, el adaptador definen cómo se almacena y recupera internamente — el CÓMO. Es un detalle de implementación invisible para el verificador externo. Cualquier implementación funcional es válida si produce resultados indistinguibles de leer las carpetas directamente.

```
# Acceso declarativo — web estática
GET turismo.gob.es/establecimientos/casapepe.es/classification/stars
→ 3

# Acceso funcional — base de datos
query({ path: 'establecimientos/casapepe.es/classification/stars' })
→ 3

# El resultado es idéntico — el parser no sabe ni le importa cómo se
almacena
```

Es el mismo principio que separa una interfaz de su implementación en programación orientada a objetos. La interfaz es declarativa — define qué existe y qué devuelve. La implementación es funcional — define cómo lo hace internamente.

11.3 El sistema de adaptadores

Un organismo como un ministerio tiene sus datos en una base de datos existente. No necesita reescribirla para participar en el protocolo — solo necesita publicar una función adaptadora que responde a rutas del protocolo con los mismos valores que tendría una carpeta.

El organismo publica en su nodo:

```
turismo.gob.es/trusteado/adapters/clasificacion_hotelera/
  name           ← 'clasificacion_hotelera'
  description    ← 'Verifica clasificación oficial de
establecimientos'
  input_schema   ← qué campos del protocolo necesita como entrada
  output_schema  ← qué campos devuelve al protocolo
  version
  cache/ttl      ← con qué frecuencia actualizar
```

La función adaptadora tiene una firma estándar:

```
async function query(path) {
  // path: ruta del protocolo
  // 'establecimientos/casapepe.es/classification/stars'

  // internamente consulta su base de datos
  // SELECT stars FROM establecimientos WHERE url = 'casapepe.es'

  // devuelve como si fuera una carpeta
  return { value: 3, timestamp: '2024-01-15' }
```

```
}
```

11.4 El repositorio central de adaptadores

Si el organismo no publica su adaptador, el repositorio central de ConfidenceNode mantiene adaptadores de comunidad para fuentes públicas conocidas. Cuando el organismo publique el suyo, tiene precedencia automáticamente — el parser siempre prefiere el adaptador del propio nodo.

```
confidencenode.org/trusteado/adapters/
  turismo.gob.es/clasificacion_hotelera/   ← mantenido por la
comunidad
  guia.michelin.com/estrellas/              ← mantenido por la
comunidad
```

11.5 La migración gradual

Un organismo puede moverse a su ritmo sin interrumpir el servicio:

```
Fase 1 — sin adaptador
  Parser lee carpetas estáticas si existen, o usa adaptador de
comunidad

Fase 2 — adaptador sobre base de datos existente
  Organismo publica query() sobre su BD actual — sin cambiar nada más

Fase 3 — web nativa del protocolo
  Organismo publica carpetas reales — el adaptador ya no es necesario
  La ruta funciona igual para todos los parsers existentes
```

La especificación declarativa es el destino. La implementación funcional es el camino. El protocolo funciona en los tres estados sin cambiar nada en los parsers que ya lo usan.

12. Cuestiones abiertas

Las siguientes cuestiones están identificadas pero deliberadamente fuera del alcance de esta versión del protocolo. Se documentan aquí para establecer que son conocidas y para orientar el trabajo futuro.

12.1 Fallo de una entidad ancla

Si la URL de una entidad desaparece de forma abrupta — quiebra repentina, pérdida del dominio — las credenciales que emitió quedan sin ancla. Para desapariciones ordenadas el protocolo facilita la transferencia de registros a un sucesor. Para desapariciones abruptas la solución es externa al protocolo — institucional o legal. El protocolo hereda ese límite honestamente.

12.2 Ámbito de las credenciales emitidas

El protocolo no impone restricciones sobre qué puede certificar un agente reconocido. Los mecanismos para declarar y verificar el ámbito de competencia de un agente se diseñarán en v0.3.

12.3 Privacidad avanzada mediante pruebas de conocimiento cero

Una implementación basada en zero-knowledge proofs permitiría demostrar la validez de una credencial sin revelar ningún elemento de la cadena. Esta es una dirección de investigación para versiones futuras.

12.4 Formato estándar de la capa de trámite

El protocolo define la capa de registro pero no el formato de los flujos entre nodos en la capa de trámite. Un estándar de interoperabilidad para estos flujos se abordará en v0.2.

12.5 Sistema de certificación de compatibilidad

Una herramienta de testing que evalúa si un nodo usa campos estándar para funciones estándar, o ha inventado equivalentes sin justificación. Los nodos pueden declarar extensiones justificadas bajo trusteado/extensions/ con una rationale y un organismo certificador. Un sistema de organismos de valoración y resolución de disputas sobre extensiones está previsto para v0.4.

12.6 Sandboxing de adaptadores

Los adaptadores ejecutan código de terceros. El protocolo debe especificar el entorno de ejecución restringido — sin acceso a red salvo a la URL del organismo, sin efectos secundarios, con timeout máximo. Web Workers y Content Security Policy son la base técnica. La especificación formal se abordará en v0.2.

FAQ

¿Por qué URLs y no DIDs o W3C Verifiable Credentials?

Los DIDs (Decentralised Identifiers) del W3C requieren infraestructura adicional — registros blockchain, métodos DID específicos — y no están anclados en ninguna presencia real preexistente. Las W3C Verifiable Credentials definen el formato del documento pero no el sistema de confianza ni cómo se establece la autoridad del emisor. Trusteando usa URLs porque las entidades ya existen con URLs propias — no hace falta crear ninguna infraestructura nueva para identificarlas. La URL es la identidad, no un puntero a ella.

¿Qué pasa si una entidad pierde su dominio?

Es el caso de fallo de entidad ancla documentado en la sección 11.1. Para mitigarlo, las entidades con alto grado de confianza deberían mantener mirrors de su espacio de identidad y registrar dominios alternativos. El protocolo facilita la transferencia ordenada pero no puede prevenir la pérdida abrupta — igual que ningún sistema de identidad existente puede hacerlo.

¿Puede cualquiera crear un nodo?

Sí. Cualquier entidad con una URL puede publicar su espacio de identidad siguiendo el protocolo sin permiso de nadie. Esto crea un nodo en nivel 1 — identidad declarada, verificable por contexto. Para alcanzar nivel 2 — identidad reconocida con verificabilidad criptográfica por terceros desconocidos — hace falta el reconocimiento del root o de otro nodo de confianza suficiente.

¿Qué diferencia hay entre esto y PGP Web of Trust?

PGP Web of Trust es una red de firmas entre personas donde la confianza se propaga por conocidos. No tiene estructura jerárquica, no está anclado en URLs públicas, no tiene dimensión temporal verificable, y no distingue entre tipos de relación. Trusteado es un Knowledge Graph estructurado donde las entidades son nodos con URLs propias, las relaciones tienen semántica explícita expresada en la estructura de carpetas, y el historial completo es auditable. PGP prueba que alguien conoce a alguien. Trusteado prueba qué es cada entidad y desde cuándo.

¿Es el root de Trusteado un punto de control centralizado?

El root es el nodo de arranque con mayor reputación inicial. No guarda estado — solo ejecuta verificaciones algorítmicas sobre información que las propias entidades publican. Si el root cae, cualquier nodo de alto grado puede realizar las mismas verificaciones porque el algoritmo es público. Con el tiempo las entidades con reputación suficiente se convierten en raíces de confianza independientes. El root es necesario para arrancar el sistema — no para que funcione una vez establecido.

¿Cómo se relaciona Trusteado con eIDAS?

eIDAS es el reglamento europeo de identidad digital — define los requisitos legales para que una firma electrónica tenga validez jurídica. Trusteado establece la evidencia criptográfica; eIDAS le da fuerza legal en jurisdicciones europeas. Son complementarios. Un nodo Trusteado respaldado por una entidad reconocida bajo eIDAS hereda su validez legal automáticamente.

Apéndice A — Vocabulario reservado del protocolo

Las siguientes carpetas tienen semántica específica en el protocolo. Cualquier implementación debe respetar estos nombres y su significado. La lista es extensible en versiones futuras — ningún nombre reservado puede ser redefinido por una implementación particular.

A.0 La carpeta raíz — trusteado/

Todo lo que el protocolo necesita vive bajo una única carpeta raíz: trusteado/. Es la única carpeta que una entidad necesita crear, mover, o subir para participar en el protocolo. Si la entidad cambia de hosting manteniendo el mismo dominio, mueve la carpeta trusteado/ al nuevo servidor y todo sigue funcionando — el entity_id no cambia porque se calcula sobre el dominio normalizado, no sobre la infraestructura.

```
casapepe.es/trusteado/     ← todo el protocolo vive aquí
```

```
  identity/
  registry/
  transactions/
```

A.1 Normalización de URL para el cálculo de entity_id

El entity_id se calcula externamente por cualquier verificador — la entidad no puede declararlo ni influir en él. Esto es una consecuencia directa de la regla de oro: la estructura declara información pero no puede imponer campos del protocolo.

La normalización es mínima y solo elimina campos técnicos del esquema URL — nunca paths, nunca componentes semánticos. Esto es crítico para evitar colisiones: si se eliminaran paths, dos entidades distintas podrían obtener el mismo entity_id.

```
normalizar(url):
  1. minúsculas
  2. eliminar prefijo www
  3. eliminar puerto estándar (443 para https, 80 para http)
  4. eliminar trailing slash
  5. eliminar query params (?...)
  6. eliminar fragmentos (#...)
  7. conservar el path completo  ← nunca eliminar paths

entity_id = SHA-256(normalizar(url))

Ejemplos:
  https://WWW.CasaPepe.ES:443/  →  https://casapepe.es
  https://casapepe.es/?ref=google  →  https://casapepe.es
  https://casapepe.es/#inicio  →  https://casapepe.es
  https://banco.es/santander/  →  https://banco.es/santander
  https://banco.es/mio/  →  https://banco.es/mio
```

Edge case documentado: si la normalización eliminara paths, 'banco.es/santander/' y 'banco.es/mio/santander/' podrían colapsar al mismo entity_id. La regla de conservar paths completos elimina esta posibilidad. Dos entidades con distinto path son siempre entidades distintas.

A.2 trusteado/identity/

Contiene todo lo que el nodo declara sobre sí mismo:

```
trusteado/identity/
  public_key          ← clave pública del nodo (base64, curva P-256)
  hash_migracion      ← hash(entity_id, clave_emergencia_1)
                        usado para migración voluntaria a nuevo
superior
                        se renueva tras cada migración completada
  hash_disputa        ← hash(entity_id, clave_emergencia_2)
                        usado para resolución de disputas
                        nunca se revela en migraciones normales
  entity_type         ← universidad | empresa | persona | servidor
| ...
  scope               ← ámbito declarado de las credenciales que emite
  root_certificate    ← certificado firmado por el root (si existe)
  dispute_level       ← nivel máximo al que este nodo puede elevar
disputas
```

A.3 trusteado/registry/

Contiene todo lo que el nodo certifica sobre otros:

```
trusteado/registry/
  old_identities/     ← cadena histórica de identidades anteriores
                        mantenida por el superior receptor, no por el
nodo
                        contiene la cadena completa, no solo el
vínculo inmediato
  desde/              ← fechas de incorporación de nodos reconocidos
  hasta/              ← fechas de baja de nodos reconocidos
  externals/          ← vínculos a sistemas externos no controlados
```

A.4 La carpeta externals/

Cualquier carpeta bajo externals/ declara que el nodo vincula algo que no controla. La semántica es opuesta al resto de carpetas — fuera de externals/, controlar la URL implica controlar el contenido. Dentro de externals/, el nodo solo garantiza la autenticidad del vínculo, no el contenido del destino.

A.5 Convención desde/ y hasta/

Las fechas de vigencia de una credencial se expresan como subcarpetas con formato ISO 8601 (YYYY-MM-DD). La presencia en desde/ sin entrada correspondiente en hasta/ indica que la credencial sigue vigente. Ambas carpetas están bajo control del superior, nunca del nodo certificado.

A.6 trusteado/identity/contact/ — opcional, recomendado para negocios

Campos de contacto público. Si existe la carpeta, los campos siguientes son los estándar reconocidos — usar campos no estándar para la misma función reduce la interoperabilidad:

```
trusteado/identity/contact/
  email                 ← RECOMENDADO
  phone                 ← RECOMENDADO
  web                   ← URL de contacto o formulario
  address/              ← dirección postal estructurada
    street
    number
    city
    region
    country             ← ISO 3166-1 alpha-2 (ES, FR, DE...)
    postal_code
  social/               ← presencias sociales
    twitter
    linkedin
    mastodon
  languages             ← idiomas en que atienden
```

A.7 trusteado/identity/location/ — opcional, recomendado para negocios físicos

```
trusteado/identity/location/
  lat                   ← OBLIGATORIO si existe location/
```

```
  lon                     ← OBLIGATORIO si existe location/
  place_id/
    externals/            ← IDs en sistemas externos — no controlados
      google_maps
      osm                 ← OpenStreetMap
      foursquare
  area/
    radius_km             ← área de servicio para entregas etc.
```

A.8 trusteado/identity/classification/ — opcional

Clasificación oficial del negocio. Si certified_by apunta a un nodo Trusteado, la verificación es criptográfica. Si apunta a una URL externa, se usa verifyExternal(). Si va bajo externals/, solo es un enlace sin verificación automática.

```
trusteado/identity/classification/
  [organismo]/            ← una entrada por organismo certificador
    stars                 ← entero 1-5
    category              ← esquema alfanumérico alternativo
    certified_by          ← URL del nodo o página que certifica
    since                 ← fecha de la clasificación
    until                 ← fecha de expiración (si aplica)
    calculated/
      verified            ← bool — verificado contra la fuente
      last_checked
      cache/ttl
```

A.9 trusteado/status/ — opcional, recomendado para negocios con horario

Estado en tiempo real. Los campos de status/ tienen TTL corto por defecto. open es siempre calculated/ — se deriva de open_at_today y close_at_today.

```
trusteado/status/
  open_at_today           ← hora apertura hoy (HH:MM) — para clientes de
hoy
  close_at_today          ← hora cierre hoy (HH:MM) — para clientes de
hoy
  schedule/               ← horario habitual — para reservas futuras
    monday/ ... sunday/
      open_at
      close_at
    exceptions/           ← festivos, vacaciones, eventos
      [YYYY-MM-DD]/
        open_at
        close_at
        closed            ← bool explícito
        note              ← 'Semana Santa', 'Vacaciones agosto'
  occupancy/
    current               ← ocupación actual (0-100%)
    capacity              ← aforo máximo
    cache/ttl             ← corto — dato en tiempo real
  queue/
    current_number        ← número que se atiende ahora
    last_issued           ← último número emitido
    wait_minutes          ← espera estimada
    cache/ttl
  special/                ← ofertas o eventos activos
```

```
    [offer_id]          ← referencia a transactions/trusteando/offers/
  calculated/
    open                ← bool: now >= open_at_today && now <
close_at_today
    open/cache/
      ttl               ← segundos hasta próximo cambio de estado
```

A.10 trusteado/notifications/ — opcional

Canal de notificaciones para parsers y agregadores. Permite mantenerse sincronizado sin recorrer el árbol completo.

```
trusteado/notifications/
  last_updated          ← timestamp del último cambio
  changelog/
    [timestamp]/
      path              ← qué carpeta cambió
      action            ← created | updated | deleted | calculated
      summary
  cache/ttl             ← corto — los parsers comprueban
frecuentemente
```

A.11 Convención cache/ — aplicable a cualquier carpeta

Cualquier carpeta puede declarar su comportamiento de caché con una subcarpeta cache/. Si no existe, se aplica el TTL por defecto del protocolo según el tipo de contenido.

```
property/cache/
  ttl                   ← segundos hasta expiración (0 = no cachear)
  strategy              ← no_cache | short | normal | long

TTL por defecto según contenido:
  status/calculated/    → no_cache (0s)
  status/occupancy/     → corto (60s)
  transactions/offers/  → corto (300s)
  identity/             → largo (86400s)
  registry/             → normal (3600s)
  notifications/        → corto (120s)
```

A.12 trusteado/adapters/ — opcional

Adaptadores que permiten a organismos con bases de datos propias exponer sus datos con la misma interfaz que las carpetas del protocolo. El organismo publica la función, el parser la llama con rutas del protocolo y recibe valores como si fueran carpetas.

```
trusteado/adapters/
  [nombre_funcion]/
    name
    description
    input_schema        ← campos del protocolo que necesita
    output_schema       ← campos que devuelve
    version
    cache/ttl
```

A.13 Campos obligatorios, recomendados y opcionales

OBLIGATORIO — sin estos campos el nodo no es válido para el protocolo:

- trusteado/identity/public_key
- trusteado/identity/hash_migracion
- trusteado/identity/hash_disputa
- trusteado/identity/entity_type
- trusteado/identity/location/lat y lon — si existe location/

RECOMENDADO — opcional pero mejora la interoperabilidad:

- trusteado/identity/scope
- trusteado/identity/root_certificate — si existe reconocimiento formal
- trusteado/identity/contact/ — para negocios
- trusteado/identity/location/ — para negocios físicos
- trusteado/status/ — para negocios con horario
- trusteado/notifications/ — para nodos con cambios frecuentes

OPCIONAL — libre, pero si se implementa debe seguir el esquema estándar:

- trusteado/identity/classification/
- trusteado/status/queue/
- trusteado/status/occupancy/
- trusteado/routing/
- trusteado/adapters/
- trusteado/features/
- trusteado/payments/
- trusteado/messaging/
- trusteado/media/
- trusteado/hooks/

Antes de crear un campo nuevo, verifica si algún campo estándar ya cumple esa función. Usar campos estándar garantiza que parsers, agregadores, y aplicaciones de terceros puedan interpretar tu nodo sin configuración adicional. Los campos no estándar son ignorados por implementaciones que no los conocen — lo que es correcto — pero se pierde interoperabilidad. Si creas un campo no estándar que aporta valor diferencial, decláralo bajo trusteado/extensions/ con una justificación.

A.14 Convención de booleanos por presencia

En el protocolo, la presencia de una carpeta indica verdadero — su ausencia indica falso. No hay campos con valor bool explícito. Esto simplifica la verificación — un parser comprueba si la carpeta existe, no si el valor es true o false. Y simplifica la publicación — el negocio crea o elimina carpetas, no edita valores.

```
trusteado/payments/accepted/
  cash/           ← existe → acepta efectivo
  card/           ← existe → acepta tarjeta
  google_pay/     ← existe → acepta Google Pay
                  ← bizum/ no existe → no acepta bizum

trusteado/features/
  parking/        ← existe → tiene parking
  wifi/           ← existe → tiene wifi
```

```
                    ← pool/ no existe → no tiene piscina
```

A.15 trusteado/features/ — opcional

Características verificables del negocio — hechos estructurados que parsers y agregadores pueden usar para filtrar y comparar. No es marketing — es información interoperable. Algunos campos pueden tener un certified_by si un organismo los verifica.

```
trusteado/features/
  parking/                ← bool por presencia
    type/                 ← free/ | paid/ | valet/
    spaces                ← número de plazas
  wifi/                   ← bool por presencia
    speed_mbps
  accessibility/          ← bool por presencia
    wheelchair/           ← bool por presencia
    elevator/             ← bool por presencia
    certified_by          ← organismo verificador opcional
  pets/                   ← bool por presencia
    conditions            ← texto libre
  pool/                   ← bool por presencia
    type/                 ← indoor/ | outdoor/ | both/
  checkin/
    from                  ← HH:MM
    until                 ← HH:MM
  checkout/
    until                 ← HH:MM
  restaurant/             ← bool por presencia
    cuisine               ← tipo de cocina
  smoking/                ← bool por presencia — permite fumar
  air_conditioning/       ← bool por presencia
  heating/                ← bool por presencia
```

A.16 trusteado/payments/ — opcional

Métodos de pago aceptados y hook al proveedor de pagos. El protocolo no implementa pagos — define dónde se conectan. Cambiar de proveedor es modificar un solo campo.

```
trusteado/payments/
  accepted/               ← métodos aceptados — booleanos por
presencia
    cash/
    card/
    contactless/
    bizum/
    google_pay/
    apple_pay/
    invoice/              ← acepta factura
  agent/                  ← hook al proveedor de pagos
    provider              ← URL del nodo del proveedor
    endpoint              ← donde se inicia la transacción
    externals/
      stripe
      redsys
      paypal
```

A.17 trusteado/messaging/ — opcional

Hook al proveedor de mensajería. El protocolo no implementa mensajería — define dónde se conecta. Se recomienda priorizar protocolos abiertos como Matrix sobre soluciones propietarias.

```
trusteado/messaging/
  agent/
    provider              ← URL del nodo del proveedor
    externals/
      whatsapp
      telegram
      matrix              ← protocolo abierto recomendado
      email               ← fallback universal
```

A.18 trusteado/hooks/ — opcional

Punto de conexión general para servicios externos. Los hooks siguen un principio deliberado: el negocio nunca debe estar atado a un proveedor específico por razones técnicas. El protocolo define los puntos de conexión — el mercado decide quién los implementa mejor. Cambiar de proveedor debe ser siempre una decisión de negocio, nunca una migración técnica.

```
trusteado/hooks/
  payments/agent/provider     ← proveedor de pagos
  messaging/agent/provider    ← proveedor de mensajería
  booking/agent/provider      ← proveedor de reservas
  delivery/agent/provider     ← proveedor de entregas
  analytics/agent/provider    ← proveedor de analíticas
```

A.19 trusteado/media/ — opcional

Imágenes y contenido multimedia verificable. Los hashes garantizan que el contenido no ha sido alterado. El almacenamiento real puede estar en cualquier CDN — el protocolo solo registra el hash y la referencia.

```
trusteado/media/
  photos/
    [photo_id]/
      hash              ← SHA-256 del fichero
      url               ← donde está almacenada la imagen
      caption           ← descripción
      taken_at          ← fecha
      certified_by      ← quien verificó que es real (opcional)
  logo/
    hash
    url
```

A.20 trusteado/rescue/ — opcional pero crítico

La carpeta de resiliencia declara dónde están las copias operativas del nodo si el servidor principal cae. Tiene una propiedad especial respecto al caché: es la única carpeta del protocolo con prioridad de caché obligatoria.

La paradoja de rescue/ es que si solo se lee cuando el servidor cae, ya es demasiado tarde para leerla. Por eso todo parser que visite un nodo debe cachear rescue/ en su primera visita y renovarla periódicamente — independientemente del TTL del resto del árbol. Un parser que no tiene rescue/ en caché no puede ofrecer resiliencia ante caídas.

```
trusteado/rescue/
  cache/
    ttl                 ← 604800s (7 días) por defecto
    priority            ← max — se cachea antes que cualquier otra
carpeta
    strategy            ← always — nunca se omite aunque el resto no
se cachee
  mirrors/              ← copias del nodo en otros servidores
    [mirror_id]/
      url               ← URL del mirror
      last_synced       ← timestamp de la última sincronización
      operator          ← quién opera el mirror
      verified/         ← bool por presencia — mirror verificado
  fallback/
    ttl                 ← cuánto tiempo es válida la copia cacheada
  backup/
    frequency           ← cada cuánto se hace backup
    last_backup         ← timestamp del último backup
    externals/
      github            ← repo donde se publica el backup
      ipfs              ← si usa IPFS para distribución
  emergency/
    contact/
      primary           ← contacto principal
      secondary         ← contacto alternativo
      protocol          ← URL al documento de procedimiento
    escalation/
      level_1           ← primer escalado si primary no responde
      level_2           ← segundo escalado
      root              ← escalar al root como último recurso
    status_page         ← URL donde se publica el estado del nodo
    hook/               ← comodín para sistemas de gestión de
emergencias
      provider          ← URL del sistema externo
      endpoint          ← donde se notifica el problema
      severity_levels/  ← niveles de severidad que maneja
      externals/        ← si usa sistemas conocidos
        pagerduty
        opsgenie
        victorops
```

Para casos de uso con requisitos de disponibilidad críticos, el hook emergency/hook/ permite conectar el protocolo con cualquier sistema de gestión de incidentes externo. El protocolo no especifica ese sistema — solo define el punto de conexión. Las extensiones necesarias para esos casos de uso se declaran bajo trusteado/extensions/ siguiendo el proceso estándar.

El orden de caché recomendado para cualquier parser:

```
1. rescue/         ← SIEMPRE — primera visita y renovación periódica
2. identity/       ← alta prioridad — cambia poco
3. notifications/  ← para detectar cambios posteriores
4. registry/       ← normal
5. status/         ← corto o no_cache
6. resto           ← según su propio cache/ttl declarado
```

A.21 trusteado/safety/ — opcional, recomendado para locales públicos

Información de seguridad del establecimiento. Tiene dos capas complementarias: información básica local — disponible desde el principio, publicada por el propio establecimiento — y referencias a agentes externos con autoridad real sobre la seguridad del local.

La presencia de fully_implemented/ junto a un agente indica que ese agente tiene una implementación completa y específica para este establecimiento en su propio nodo. Su ausencia indica que la información local puede ser la única disponible.

```
trusteado/safety/

  ← Información básica local
  emergency_exits/
    [exit_id]/
      location          ← descripción o coordenadas
      floor
      accessible/       ← bool por presencia
  assembly_point        ← punto de encuentro exterior
  capacity/
    max                 ← aforo máximo legal
    certified_by
  defibrillator/        ← bool por presencia
    location
  first_aid/            ← bool por presencia
    location
  evacuation/
    plan_url

  ← Referencias a agentes con autoridad real
  authorities/
    fire/
      node              ← URL nodo bomberos con entrada específica
      fully_implemented/ ← bool por presencia — plan completo
verificable
      externals/phone   ← fallback mientras no hay nodo completo
    police/
      node
      fully_implemented/
      externals/phone
    medical/
      node
      fully_implemented/
      externals/phone
    food_safety/
      node
      fully_implemented/
    building/
      node              ← ayuntamiento — licencias y normativa
edilicia
      fully_implemented/
    insurance/
      node              ← aseguradora — póliza activa
      fully_implemented/
```

La seguridad de un local no es lo que el local dice sobre sí mismo — es la red de organismos con autoridad que han implementado algo específico sobre él. La información local bajo safety/ es siempre útil como punto de partida y como redundancia. La presencia de fully_implemented/ junto a un agente indica que la fuente autoritativa está en el nodo externo y que el trabajo previo está completado y verificable.

El protocolo define una jerarquía de resolución de disputas análoga al recurso de alzada en derecho administrativo. Las disputas se resuelven al nivel más bajo posible. Elevar una disputa tiene un coste creciente para desincentivar el uso frívolo.

B.1 Jerarquía de resolución

```
Nivel 1 — el superior inmediato
  Juan tiene disputa con la universidad
  → la universidad resuelve usando hash_disputa de Juan
  Coste: bajo

Nivel 2 — recurso de alzada
  Juan no acepta la resolución de la universidad
  → eleva al nodo superior de la universidad
  → ese nodo resuelve con su propia autoridad
  Coste: medio

...

Nivel N — el root como tribunal de última instancia
  Solo para disputas que han agotado niveles inferiores
  O para disputas entre nodos sin superior común
  Coste: elevado
```

B.2 El coste de disputa

Cada nivel tiene un coste creciente — económico y de reputación. Elevar una disputa al root sin haber pasado por los niveles inferiores no está permitido salvo casos excepcionales tasados. Perder una disputa que se elevó innecesariamente tiene consecuencias para el nodo que la elevó. El modelo es análogo a las tasas judiciales: existen para que los recursos sean proporcionales a la gravedad del asunto.

B.3 El rol de hash_disputa

El hash_disputa — derivado de clave_emergencia_2, nunca revelado en migraciones normales — es la prueba de identidad ante el árbitro. En una disputa el nodo demuestra que conoce la clave que produce ese hash sin revelarla. El árbitro verifica la prueba y resuelve. Si la disputa sube de nivel, el hash_disputa acompaña el expediente — es la cadena de custodia criptográfica de la identidad del disputante.

Apéndice C — API mínima del protocolo

La API define cómo se interactúa con un nodo. Es independiente de la implementación — un nodo puede ser un servidor web estático, una API REST, o un microservicio, mientras exponga estos métodos con la misma semántica. La API es una conveniencia para facilitar la interoperabilidad — lo que no esté aquí puede implementarse libremente sin romper el protocolo.

C.1 Verificación

```
verify(url, folder)
  → ¿está este nodo en esta carpeta ahora?
  → devuelve: { valid: bool, timestamp: ISO8601, issuer: url }

getIdentity(url)
  → obtener trusteado_identity/ de un nodo
  → devuelve: { public_key, hash_migracion, hash_disputa,
                entity_type, scope, root_certificate }
```

C.2 Ciclo de vida

```
create(url, entity_type, scope)
  → crear un nuevo nodo en el sistema
  → genera claves, publica trusteado_identity/

certify(subject_url, folder)
  → añadir un nodo a una carpeta del registro
  → equivale a emitir una credencial

revoke(subject_url, folder)
  → eliminar un nodo de una carpeta del registro
  → equivale a revocar una credencial específica
```

C.3 Migración

```
migrate(new_superior_url, emergency_key_1)
  → solicitar migración a un nuevo superior
  → el nodo revela emergency_key_1 al nuevo superior

adoptNode(subject_url, emergency_key_1)
  → el superior acepta la migración
  → verifica emergency_key_1 contra hash_migracion publicado
  → publica old_identities/ con cadena completa
  → aprueba ante el root
```

C.4 Privacidad contextual

```
requestReveal(subject_url, field, context)
  → el receptor solicita acceso a un campo protegido
  → field: ruta de la carpeta dont-want-to-reveal/
  → context: propósito declarado de la solicitud
  → devuelve: { request_id, status: pending }

grantReveal(request_id, disclosure_level)
  → el sujeto concede acceso al campo solicitado
  → disclosure_level: RESULT_ONLY | ISSUER_ONLY |
                      CONTENT | FULL_CHAIN
  → transferable: bool  ← si el receptor puede demostrarlo a terceros

denyReveal(request_id)
  → el sujeto deniega la solicitud
```

Apéndice D — Capa de negocio — transactions/trusteando/

El protocolo Trusteado es general — sirve para identidad, certificaciones, y cualquier hecho verificable. La capa de negocio es una extensión opcional que estandariza cómo se expresan transacciones y reputación verificables. Cualquier nodo que quiera participar en el ecosistema de transacciones verificables adopta la convención transactions/trusteando/.

La convención es voluntaria pero estratégicamente importante: cualquier sistema que quiera indexar, agregar, o verificar transacciones y reputación de forma interoperable tiene que hablar este lenguaje. La carpeta es la declaración de participación — no hace falta registro central ni API especial.

D.1 Estructura de transactions/trusteando/

```
casapepe.es/transactions/trusteando/

  visits/                               ← visitas verificadas
    desde/YYYY-MM-DD/[visitor_hash]

  reviews/                              ← colección de reseñas
    review/                             ← objeto reseña (datos
primarios)
      set_id                            ← identificador único
      type                              ← entero enumerado (ver D.2)
      content_hash                      ← SHA-256 del texto
      rating                            ← valoración numérica
      visit_ref                         ← referencia a visita (si type
>= 2)
      payment_ref                       ← referencia a pago (si type >=
3)
      dont-want-to-reveal/              ← campos privados opcionales
    calculated/                         ← índices derivados
automáticamente
      verified_visit/                   ← reviews con type >= 2
      verified_paid/                    ← reviews con type >= 3
      verified_identity/                ← reviews con type >= 1
      by_rating/                        ← índice por valoración

  offers/
    [offer_id]/
      terms
      valid/desde/hasta/
      eligibility/

  reputation/
    calculated/
      score                             ← agregado calculado
      total_reviews                     ← conteo de set_id únicos
      total_verified_visits             ← conteo type >= 2
      total_verified_paid               ← conteo type >= 3
```

D.2 Tipo enumerado de reseña

El campo type es un entero enumerado donde el orden refleja el nivel de verificación — a mayor número, mayor garantía criptográfica:

type	nombre	garantía
0	unverified	sin verificación — cualquiera puede opinar
1	verified_identity	identidad del autor verificada
2	verified_visit	presencia física verificada
3	verified_paid	compra verificada por TPV o token de pago
4	verified_visit_and_paid	presencia física + compra verificadas

El orden importa para consultas: reviews con type >= 2 devuelve solo reseñas con presencia física verificada. Contar reseñas únicas es siempre contar set_id distintos en reviews/review/ — los índices en calculated/ son vistas derivadas, no duplicados.

D.3 La carpeta calculated/

Todo lo que está bajo calculated/ es derivado automáticamente de los datos primarios y verificable independientemente por cualquier tercero. Nunca se escribe a mano. El protocolo define dos funciones para gestionar campos calculados:

```
updateIndex(entity_url, collection)
  → recalcula los índices de una colección
  → se llama tras cada create/update/delete de un objeto
  → idempotente — ejecutarlo dos veces da el mismo resultado

checkIndex(entity_url, collection)
  → verifica que los índices coinciden con los datos primarios
  → devuelve: { valid: bool, mismatches: [...] }
  → cualquier tercero puede ejecutarlo — auditoría pública autónoma
```

D.4 La carpeta routing/

Declara explícitamente a quién envía datos una entidad y para qué. Es transparencia activa — cualquier tercero puede ver exactamente qué procesadores tienen autorización. Compatible con el registro de actividades de tratamiento del GDPR.

```
trusteado/routing/
  processors/           ← entidades autorizadas a procesar mis datos
    confidencenode.org  ← el root — verificaciones
    analytics.es        ← procesador de analíticas autorizado
  subscribers/          ← entidades que reciben notificaciones de
eventos
    malaga.org          ← agregador regional suscrito
  webhooks/             ← endpoints que reciben eventos
automáticamente
```

D.5 Semántica de los prefijos reservados

carpeta	semántica
`externals/`	enlace a algo no controlado por el nodo
`calculated/`	derivado automáticamente, verificable con checkIndex()
`dont-want-to-reveal/`	existe pero no expuesto por defecto — grantReveal() para acceso
`routing/`	procesadores y suscriptores autorizados a recibir datos
`fully_implemented/`	bool por presencia — el nodo externo referenciado tiene implementación completa y específica. Su ausencia indica implementación parcial o en progreso — la información local puede ser la única disponible.

D.6 La propiedad central

Una reseña bajo transactions/trusteando/reviews/review/ no es una opinión — es una arista del grafo que conecta un nodo verificado con una visita o compra verificada. El contenido puede ser subjetivo pero la presencia es objetiva. Cualquier verificador puede comprobarlo consultando URLs públicas.

Esta propiedad destruye las reseñas falsas por construcción, no por moderación. Y la auditoría es pública y autónoma — cualquiera puede ejecutar checkIndex() y confirmar que los índices de reputación son consistentes con los datos primarios.

D.7 Valor estratégico del nombre

La convención transactions/trusteando/ asocia el nombre Trusteando con el estándar de transacciones verificables del protocolo. Cualquier implementación que adopte esta convención refuerza ese vínculo. El dominio trusteando.app es la implementación de referencia — el punto de entrada natural para cualquiera que busque qué significa esta carpeta o cómo implementarla.

Apéndice E — Diccionario de intencionalidad — inspiración en mecanismos web

Trusteando toma inspiración de mecanismos web establecidos. El diccionario siguiente ayuda a entender la intencionalidad de cada carpeta — no implica equivalencia técnica ni compatibilidad directa. Donde la web resolvió problemas similares, Trusteado añade firma criptográfica, semántica de identidad, y verificabilidad pública.

Trusteado	Inspirado en	Diferencia clave
`trusteado/redirect/`	HTTP 301/302	Firmado, forma parte del grafo de identidad, con fecha y cadena histórica
`trusteado/notifications/`	HTTP headers / ETag	Declarativo en carpeta, verificable, sin negociación de cabeceras
`notifications/`	RSS / Atom	Sin servidor de feed, pull directo sobre

`changelog/`		carpetas públicas
`trusteado/routing/`	robots.txt / CORS	Con semántica de autorización firmada y compatible con GDPR
`trusteado/registry/`	sitemap.xml	Con firmas criptográficas y semántica de credencial
`calculated/`	Cache / ETag	Auditable públicamente con checkIndex() — no solo para rendimiento
`externals/`	<a href> externo	Declara explícitamente falta de control sobre el destino
`routing/processors/`	GDPR registro tratamiento	Verificable criptográficamente, no solo declarativo
`dont-want-to-reveal/`	— (sin equivalente)	Privacidad contextual con divulgación selectiva — no existe en la web

La carpeta trusteado/redirect/ merece mención especial. A diferencia de un HTTP 301, un redirect en Trusteado es una declaración firmada de migración de identidad — con fecha, con referencia a la clave de emergencia usada, y formando parte del grafo histórico del nodo. No es solo una instrucción técnica de enrutamiento sino un hecho verificable con las mismas garantías que cualquier otra credencial del protocolo.

```
trusteado/redirect/
  target          ← URL de destino (nueva identidad)
  permanent       ← bool — migración definitiva o temporal
  since           ← fecha de la migración
  migration_ref   ← referencia al proceso de migración en
old_identities/
```

Referencias

- ECDH — Elliptic Curve Diffie-Hellman: RFC 7748, Bernstein et al., 2016
- HMAC — Hash-based Message Authentication Code: RFC 2104, Krawczyk et al., 1997
- SHA-256 — Secure Hash Algorithm: FIPS PUB 180-4, NIST, 2015
- W3C Verifiable Credentials Data Model: W3C Recommendation, 2022
- W3C Decentralised Identifiers (DIDs): W3C Recommendation, 2022
- TLS 1.3 — Transport Layer Security: RFC 8446, Rescorla, 2018
- eIDAS — Reglamento (UE) 910/2014 sobre identificación electrónica, 2014
- GNU General Public License v3: Free Software Foundation, 2007

Autor: confidencenode.org/members/confidencenode0 — confidencenode.org/protocolos/trusteando

18 de Marzo de 2026 — v0.1 — ConfidenceNode — confidencenode.org

Trusteando es software libre licenciado bajo GNU GPL v3.

El protocolo es abierto. Cualquiera puede implementarlo, verificarlo, y construir sobre él.

trusteando.app — github.com/ConfidenceNode

Autor: confidencenode.org/members/confidencenode0 — confidencenode.org/protocolos/trusteando

18 de Marzo de 2026 — v0.1 — ConfidenceNode — confidencenode.org

Trusteando es software libre licenciado bajo GNU GPL v3.

El protocolo es abierto. Cualquiera puede implementarlo, verificarlo, y construir sobre él.

trusteando.app — github.com/ConfidenceNode